CAP455 Minor Project - CA-1 (Progress Report)

TOPIC: Dairy Farm Milk Quality and Collection.

Name: Tania
Reg No: 12616112
Section: D1P2634

1. Project Proposal:

1. Project Overview & Background

In rural and semi-urban cooperative dairy setups, local farmers bring fresh milk twice a day to village-level collection centers. Before this milk is poured into communal refrigerated storage silos, every supply must be measured for volume and checked for quality—specifically its Fat percentage and Solids-Not-Fat (SNF) levels. This quality check is critical because it directly decides the payout given to the farmer and ensures adulterated or sub-standard milk doesn't spoil the whole bulk batch.

Right now, many small collection centers still rely on paper registers and basic hand calculators. Staff members manually write down the farmer's name, measure milk volume with dipsticks or graduated cans, test a small sample using a butyrometer, and calculate payouts on paper. This manual approach frequently leads to billing errors, duplicate serial records, and long waiting queues during early morning milking hours.

2. Problem Statement

Manual ledger-based management at dairy intake counters causes several operational bottlenecks:

- **Duplicate Entries & ID Clashes:** Busy operators often duplicate batch numbers or confuse farmer account numbers during peak collection hours.
- **Calculation Inaccuracies:** Quality-indexed payout calculations (based on variable Fat and SNF tiers) are error-prone when calculated manually under time pressure.
- **Zero Real-Time Visibility:** Supervisors have no immediate way to know how much milk has entered the chiller tank at any given moment without manually adding up dozens of register pages.

3. Proposed Solution

This project develops a console-based software system in C++ tailored for milk procurement booths. The system digitizes the intake counter workflow by letting the operator log each incoming batch with automatic duplicate-key validation, instant payout calculation, and running tallies of storage vat capacity. The design uses standard Object-Oriented Programming (OOP) principles to keep the codebase clean, modular, and easy to maintain.

4. Core Objectives

For the CA1 milestone, the project implements the primary in-memory foundation:

- **Primary Key Verification:** Enforce unique batch identification numbers before any new record is created to prevent double-billing or accidental overwrites.

- **Automated Pricing Logic:** Implement instant rate-per-liter calculation tied directly to fat percentage standards so payouts are computed transparently.
- **Quality Range Validation:** Reject records that fail realistic biological standards (e.g., rejecting impossible fat percentages below 3.0% or above 12.0%).
- **Real-Time Storage Aggregation:** Maintain shared class-level counters to track total volume collected in the cooling tank during the current session.
- **External Auditing Slip Generation:** Provide an independent verification slip utility that displays complete batch metrics for quality auditors.

5. Technical Stack & OOP Design Mapping

- **Language & Standard:** C++ (C++11 / C++14)
- **Storage Model:** In-memory array of objects managed via pointers for high-speed sequential lookups.
- **Core OOP Constructs Applied:**
  - `Entity Class (MilkCollection)`: Encapsulates individual intake attributes such as farmer identity, milk type (Cow/Buffalo), volume, Fat %, SNF %, and calculated amounts.
  - `static Members & Methods`: Shared counters (`totalBatches`, `totalMilkProcured`) track center-wide statistics without duplicating data across separate farmer objects.
  - `inline Function`: Computes financial payouts inline to eliminate function call overhead during rapid data entry.
  - `friend Function`: Simulates an external dairy inspector module that can read raw private batch attributes to print tamper-proof audit slips.
  - `this Pointer`: Resolves member variable naming collisions inside initialization routines and handles explicit object references.

6. Expected Outcomes & Future Scope

Completing this phase yields a working prototype that automates the check-in and quality appraisal of incoming milk batches. In subsequent phases, this foundational architecture will be expanded with persistent binary file storage, hierarchical farmer/vendor categorization and safe record deletions, and full exception handling with STL container integration.

2. Data Dictionary

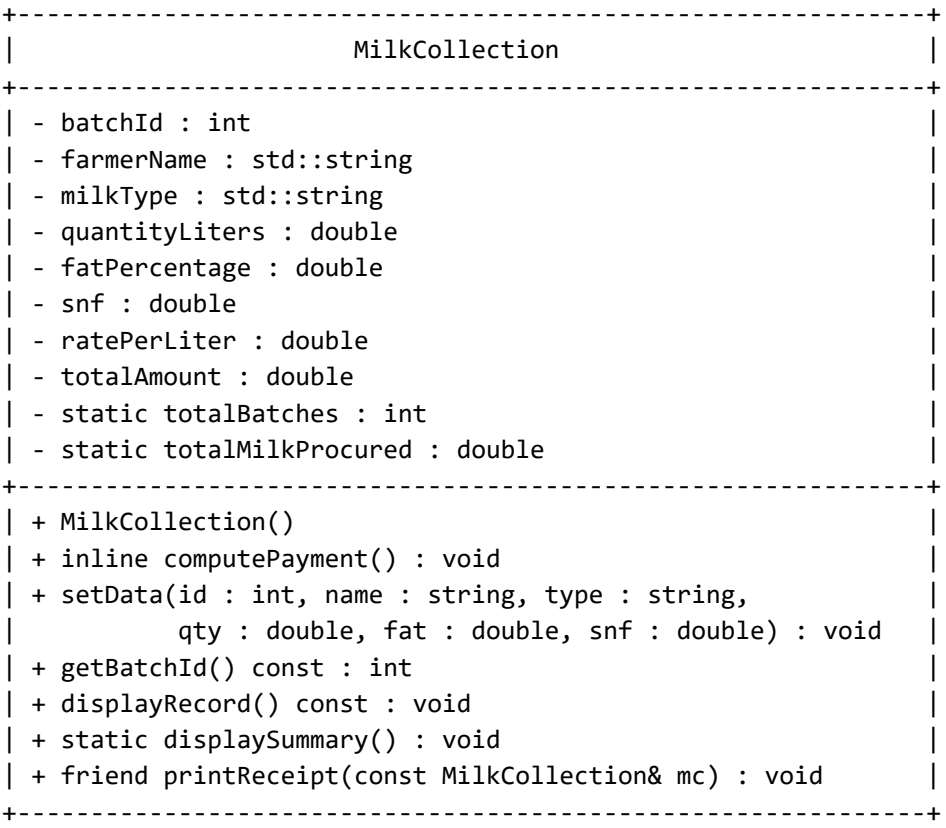
The entity class `MilkCollection` manages individual milk intake records:

Attribute / Field	Data Type	Visibility	Validation / Constraints	Purpose / Description
batchId	int	private	Positive integer, strictly unique	Unique identification key for each collection entry
farmerName	std::string	private	Non-empty string	Name of the supplying farmer
milkType	std::string	private	"Cow" or "Buffalo"	Type of milk supplied
quantityLiters	double	private	> 0.0 and <= 1000.0	Volume of milk received in liters
fatPercentage	double	private	3.0 to 12.0	Fat content measured via milk analyzer / butyrometer
snf	double	private	7.0 to 10.5	Solids-Not-Fat percentage for purity checking
ratePerLiter	double	private	Computed: fatPercentage * 8.5	Standard quality-indexed rate per liter in ₹
totalAmount	double	private	Computed: quantityLiters * ratePerLiter	Payout generated for the batch
totalBatches	int (static)	private	Increments on valid entry	Class-wide count of batches logged
totalMilkProcured	double (static)	private	Cumulative volume	Total volume of milk accumulated in vat

The array holds at most 50 records in this phase. And SNF is checked but does not change the price yet, and the milk type does not change the rate either. I kept the pricing rule simple for CA-1 and plan to refine it in a later phase.

3. UML Class Diagram (Textual Representation)

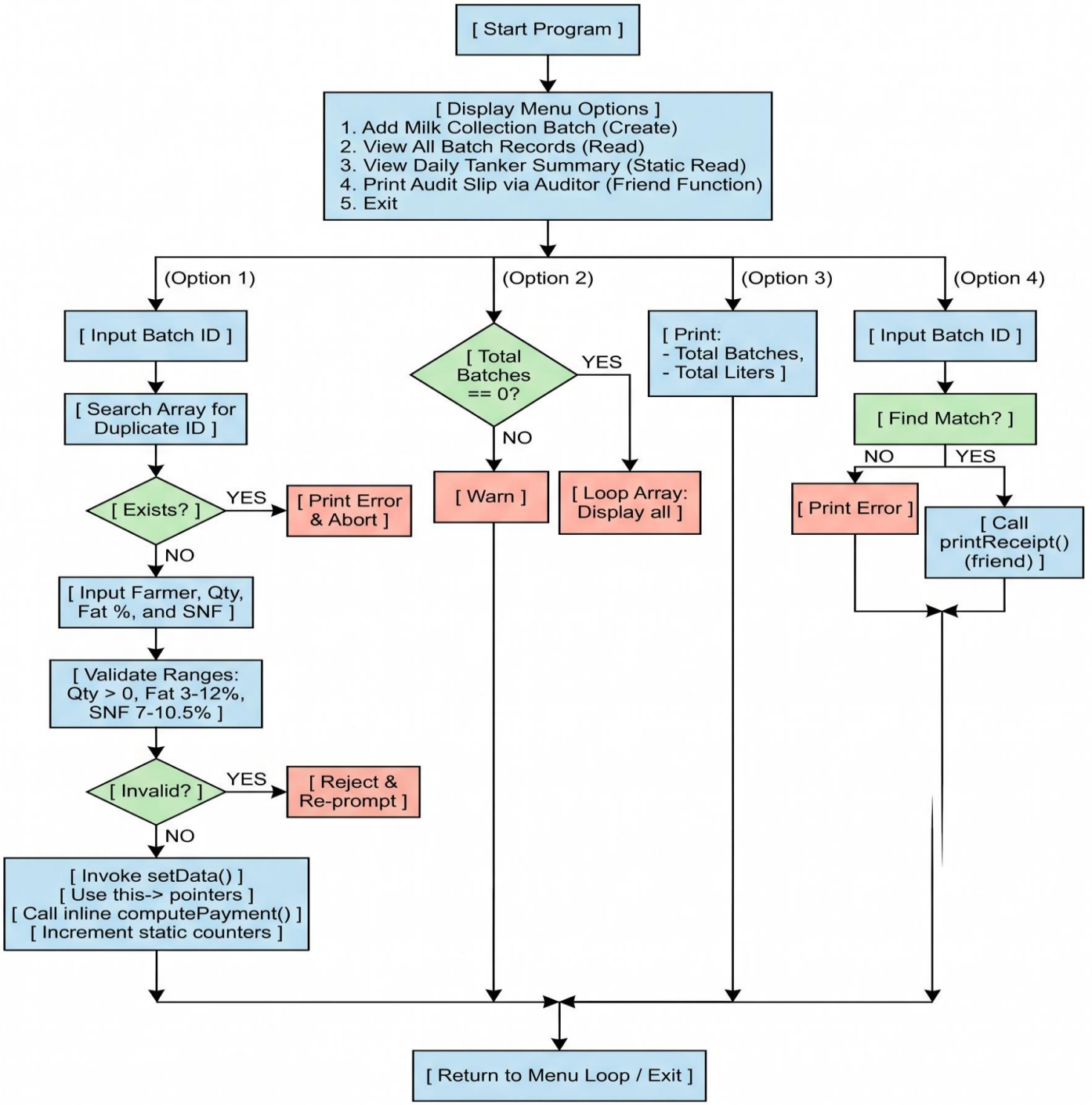
The whole design is one class. A minus sign means private, a plus sign means public, and static members belong to the class and not to a single object.

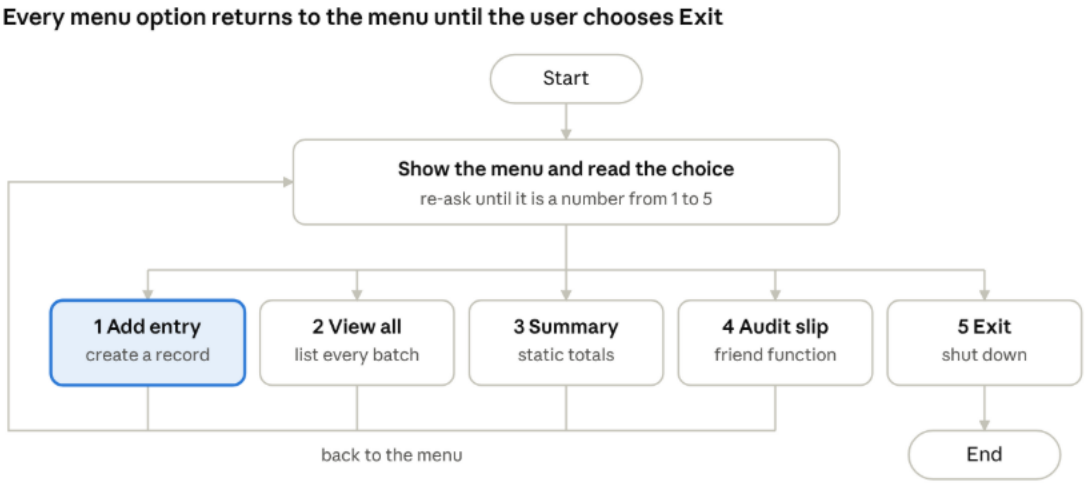


Outside the class there are a few ordinary helper functions: isDuplicateKey, readInt, readDouble, readNonEmptyLine and main. The friend function printReceipt is not a member, but the class allows it to read the private fields, which is how the auditor slip gets the batch details.

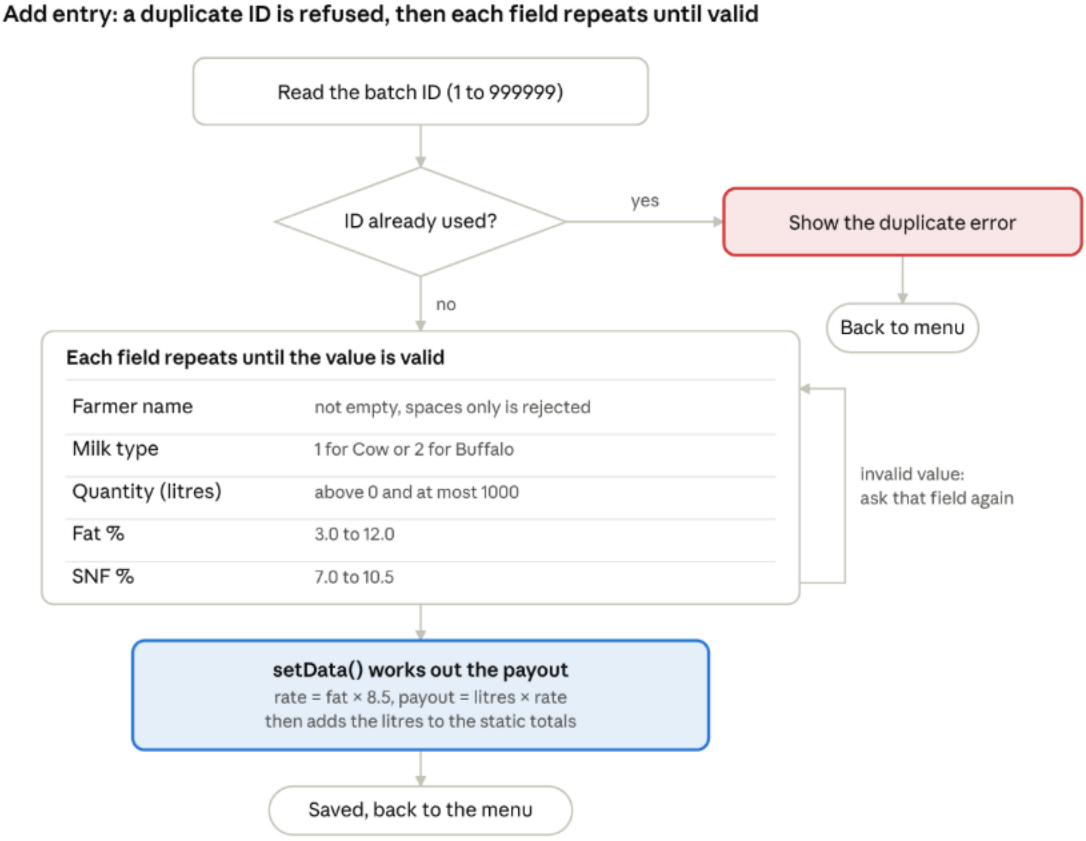
4. Logic Flowchart

First the big picture. The program shows a menu, runs the option the user picks, and comes back to the menu until the user chooses Exit.





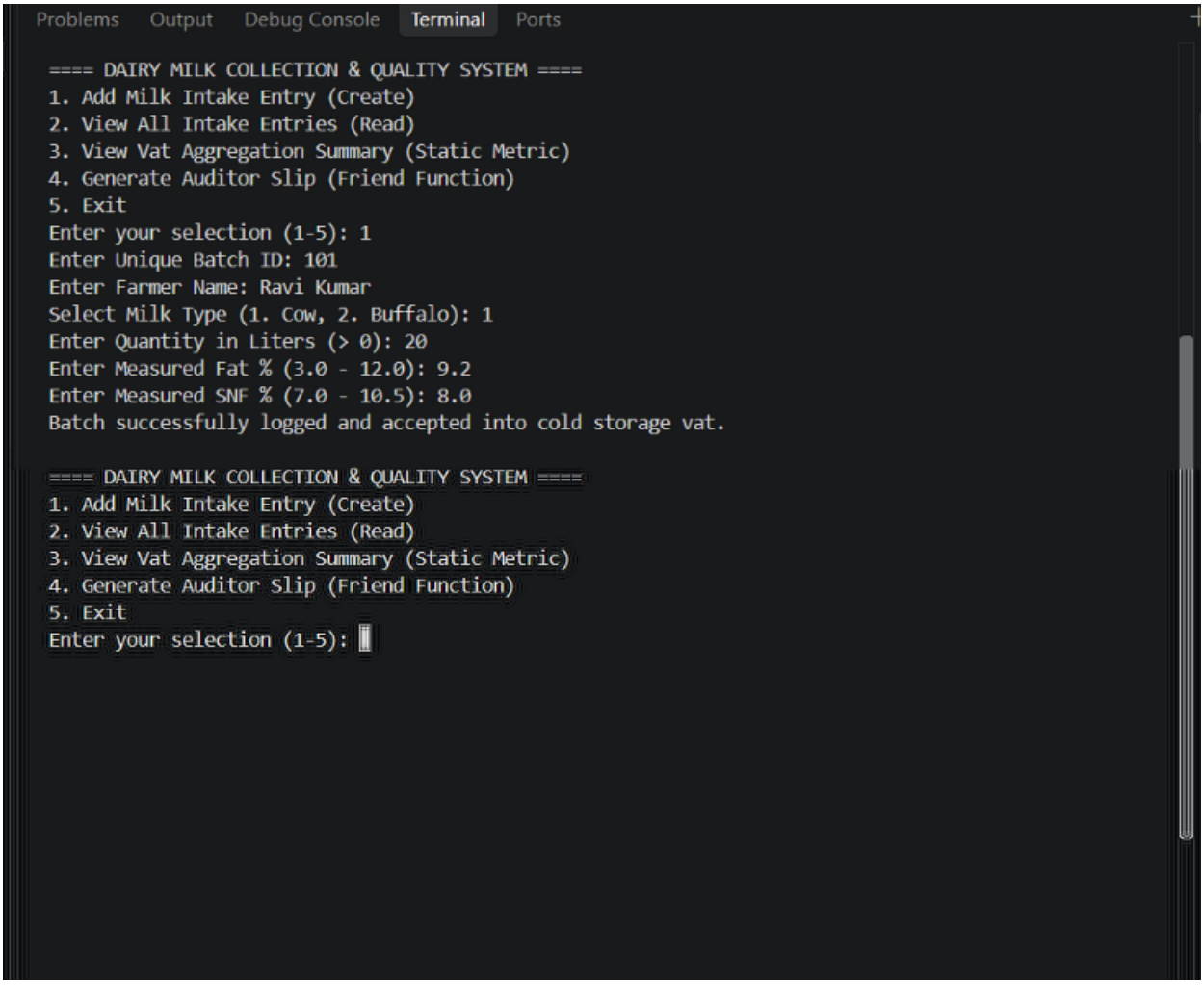
It refuses a duplicate batch ID straight away, then asks for each field again until the value is valid, and only then stores the record and updates the totals.



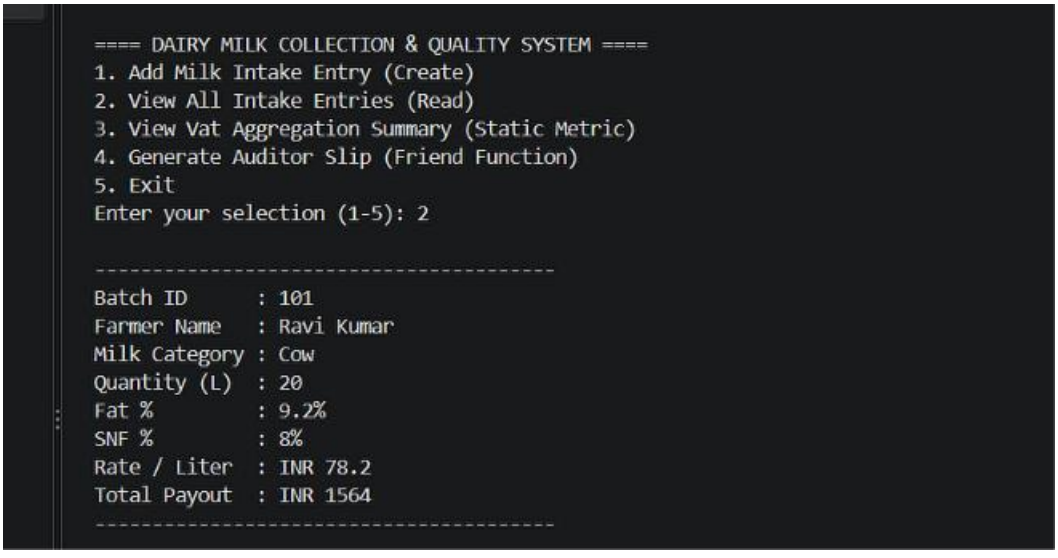
Option 2 prints a short message if nothing is stored yet, otherwise it walks through the array with a pointer and displays every record. Option 4 asks for a batch ID, prints the auditor slip if the batch exists, and says it was not found if it does not.

5. Terminal Screenshots

1. Entering the data.



2. Viewing all the entries.



3. Viewing Vat aggregation summary.

4. Generating Auditor slip.

```
==== DAIRY MILK COLLECTION & QUALITY SYSTEM ====
1. Add Milk Intake Entry (Create)
2. View All Intake Entries (Read)
3. View Vat Aggregation Summary (Static Metric)
4. Generate Auditor Slip (Friend Function)
5. Exit
Enter your selection (1-5): 3

========================================
     DAIRY COLLECTION TANKER SUMMARY
========================================
Total Batches Logged : 1
Total Milk in Vat    : 20 Liters
========================================

==== DAIRY MILK COLLECTION & QUALITY SYSTEM ====
1. Add Milk Intake Entry (Create)
2. View All Intake Entries (Read)
3. View Vat Aggregation Summary (Static Metric)
4. Generate Auditor Slip (Friend Function)
5. Exit
Enter your selection (1-5): 4
Enter Batch ID for Auditor Verification: 101

========================================
        DAIRY AUDITOR OFFICIAL SLIP
========================================
Verified Batch: #101
Producer      : Ravi Kumar (Cow)
Volume Intake : 20 L
Fat / SNF     : 9.2% / 8%
Net Credited  : INR 1564
Quality Status: APPROVED FOR VAT STORAGE
========================================
```

5. Exiting.

```
==== DAIRY MILK COLLECTION & QUALITY SYSTEM ====
1. Add Milk Intake Entry (Create)
2. View All Intake Entries (Read)
3. View Vat Aggregation Summary (Static Metric)
4. Generate Auditor Slip (Friend Function)
5. Exit
Enter your selection (1-5): 5
Shutting down collection portal.
PS C:\CAP455\dairy>
```

6. Source Code

```
#include <iostream>
#include <string>
class MilkCollection {
private:
    int batchId;
    std::string farmerName;
    std::string milkType;
    double quantityLiters;
    double fatPercentage;
    double snf;
    double ratePerLiter;
    double totalAmount;
    static int totalBatches;
    static double totalMilkProcured;
public:
    MilkCollection() {
        this->batchId = 0;
        this->farmerName = "";
        this->milkType = "";
        this->quantityLiters = 0.0;
        this->fatPercentage = 0.0;
        this->snf = 0.0;
        this->ratePerLiter = 0.0;
        this->totalAmount = 0.0;
    }
    inline void computePayment() {
        this->ratePerLiter = this->fatPercentage * 8.5;
        this->totalAmount = this->quantityLiters * this->ratePerLiter;
    }
    void setData(int batchId, std::string farmerName, std::string milkType,
                 double quantityLiters, double fatPercentage, double snf) {
        this->batchId = batchId;
        this->farmerName = farmerName;
        this->milkType = milkType;
        this->quantityLiters = quantityLiters;
        this->fatPercentage = fatPercentage;
        this->snf = snf;
        this->computePayment();
        totalBatches++;
        totalMilkProcured += this->quantityLiters;
    }
    int getBatchId() const {
        return this->batchId;
    }
    void displayRecord() const {
```

```
        std::cout << "\n----------------------------------------\n";
        std::cout << "Batch ID      : " << this->batchId << "\n";
        std::cout << "Farmer Name   : " << this->farmerName << "\n";
        std::cout << "Milk Category : " << this->milkType << "\n";
        std::cout << "Quantity (L)  : " << this->quantityLiters << "\n";
        std::cout << "Fat %         : " << this->fatPercentage << "%\n";
        std::cout << "SNF %         : " << this->snf << "%\n";
        std::cout << "Rate / Liter  : INR " << this->ratePerLiter << "\n";
        std::cout << "Total Payout  : INR " << this->totalAmount << "\n";
        std::cout << "----------------------------------------\n";
    }
    static void displaySummary() {
        std::cout << "\n========================================\n";
        std::cout << "     DAIRY COLLECTION TANKER SUMMARY    \n";
        std::cout << "========================================\n";
        std::cout << "Total Batches Logged : " << totalBatches << "\n";
        std::cout << "Total Milk in Vat    : " << totalMilkProcured << " Liters\n";
        std::cout << "========================================\n";
    }
    friend void printReceipt(const MilkCollection &mc);
};
int MilkCollection::totalBatches = 0;
double MilkCollection::totalMilkProcured = 0.0;
void printReceipt(const MilkCollection &mc) {
    std::cout << "\n========================================\n";
    std::cout << "        DAIRY AUDITOR OFFICIAL SLIP     \n";
    std::cout << "========================================\n";
    std::cout << "Verified Batch: #" << mc.batchId << "\n";
    std::cout << "Producer      : " << mc.farmerName << " (" << mc.milkType << ")\n";
    std::cout << "Volume Intake : " << mc.quantityLiters << " L\n";
    std::cout << "Fat / SNF     : " << mc.fatPercentage << "% / " << mc.snf << "%\n";
    std::cout << "Net Credited  : INR " << mc.totalAmount << "\n";
    std::cout << "Quality Status: APPROVED FOR VAT STORAGE\n";
    std::cout << "========================================\n";
}
bool isDuplicateKey(MilkCollection records[], int count, int newId) {
    for (int i = 0; i < count; i++) {
        if (records[i].getBatchId() == newId) {
            return true;
        }
    }
    return false;
}
int main() {
    const int MAX_RECORDS = 50;
    MilkCollection records[MAX_RECORDS];
    MilkCollection* ptrRecords = records;
    int recordCount = 0;
    int choice = 0;
```

```
    while (choice != 5) {
        std::cout << "\n==== DAIRY MILK COLLECTION & QUALITY SYSTEM ====\n";
        std::cout << "1. Add Milk Intake Entry (Create)\n";
        std::cout << "2. View All Intake Entries (Read)\n";
        std::cout << "3. View Vat Aggregation Summary (Static Metric)\n";
        std::cout << "4. Generate Auditor Slip (Friend Function)\n";
        std::cout << "5. Exit\n";
        std::cout << "Enter your selection (1-5): ";
        std::cin >> choice;
        if (choice == 1) {
            if (recordCount >= MAX_RECORDS) {
                std::cout << "Collection batch storage limit reached.\n";
                continue;
            }
            int id;
            std::cout << "Enter Unique Batch ID: ";
            std::cin >> id;
            if (isDuplicateKey(records, recordCount, id)) {
                std::cout << "Error: Batch ID " << id << " already exists in system.
Duplicate rejected.\n";
                continue;
            }
            std::string name;
            std::cout << "Enter Farmer Name: ";
            std::cin.ignore();
            std::getline(std::cin, name);
            int typeChoice;
            std::string type = "Cow";
            std::cout << "Select Milk Type (1. Cow, 2. Buffalo): ";
            std::cin >> typeChoice;
            if (typeChoice == 2) {
                type = "Buffalo";
            }
            double qty;
            std::cout << "Enter Quantity in Liters (> 0): ";
            std::cin >> qty;
            if (qty <= 0.0 || qty > 1000.0) {
                std::cout << "Invalid volume entered. Batch rejected.\n";
                continue;
            }
            double fat;
            std::cout << "Enter Measured Fat % (3.0 - 12.0): ";
            std::cin >> fat;
            if (fat < 3.0 || fat > 12.0) {
                std::cout << "Fat percentage outside acceptable milk standard limits (3.0 -
12.0).\n";
                continue;
            }
            double snf;
```

```
            std::cout << "Enter Measured SNF % (7.0 - 10.5): ";
            std::cin >> snf;
            if (snf < 7.0 || snf > 10.5) {
                std::cout << "SNF percentage outside acceptable purity limits (7.0 -
10.5).\n";
                continue;
            }
            (ptrRecords + recordCount)->setData(id, name, type, qty, fat, snf);
            recordCount++;
            std::cout << "Batch successfully logged and accepted into cold storage vat.\n";
        } else if (choice == 2) {
            if (recordCount == 0) {
                std::cout << "No milk intake records recorded yet.\n";
            } else {
                for (int i = 0; i < recordCount; i++) {
                    (ptrRecords + i)->displayRecord();
                }
            }
        } else if (choice == 3) {
            MilkCollection::displaySummary();
        } else if (choice == 4) {
            if (recordCount == 0) {
                std::cout << "No collection records available for audit.\n";
                continue;
            }
            int searchId;
            std::cout << "Enter Batch ID for Auditor Verification: ";
            std::cin >> searchId;
            bool found = false;
            for (int i = 0; i < recordCount; i++) {
                if ((ptrRecords + i)->getBatchId() == searchId) {
                    printReceipt(*(ptrRecords + i));
                    found = true;
                    break;
                }
            }
            if (!found) {
                std::cout << "Batch ID not found in intake log.\n";
            }
        } else if (choice == 5) {
            std::cout << "Shutting down collection portal.\n";
        } else {
            std::cout << "Invalid choice. Please select from 1 to 5.\n";
        }
    }
    return 0;
}
```

7. Concepts Used and Where

This table lists each concept and the exact place in the program where I used it, so it is easy to point to during the demo.

Concept	Where it appears in the code
Classes and objects	class MilkCollection; the objects live in records[50]
Static data members and static function	totalBatches, totalMilkProcured, and displaySummary(), called as MilkCollection::displaySummary()
Inline function	computePayment() is declared inline (functions written inside the class body are also inline automatically)
Friend function	printReceipt(const MilkCollection &mc) reads the private fields to print the auditor slip
Reference variables	printReceipt takes the object by const reference, and the input helpers take const std::string& prompt
User-defined functions	isDuplicateKey, readInt, readDouble, readNonEmptyLine
Input and output streams	cin, cout, getline, cin.clear(), cin.ignore()
Array of objects	MilkCollection records[MAX_RECORDS] with MAX_RECORDS = 50
Pointer to objects and pointer arithmetic	MilkCollection* ptrRecords = records; then (ptrRecords + recordCount)->setData(...) and (ptrRecords + i)->displayRecord()
this pointer	the constructor, setData, computePayment, displayRecord and getBatchId all use this->
Array passed to a function	isDuplicateKey(MilkCollection records[], int count, int newId)
Standard string class	farmerName, milkType, std::getline, and trimming spaces with find_first_not_of and substr

8. What I learned from this phase

Building this made object-oriented programming feel much less abstract. Putting the farmer's details and the payout calculation inside one class meant I never had to carry six loose variables around, and the static counters showed me how a class can keep information that belongs to all objects instead of to any single one.

The part that taught me the most was input handling. My first draft worked fine when I typed what I was supposed to type, but a review of it showed that typing a letter where a number was expected made the menu loop forever. The reason is that cin stays in a failed state until you call cin.clear() and throw away the bad text with cin.ignore(). I rewrote the input into three small helper functions that keep asking until the value is valid, and I tested the edges of every range, for example 1000 litres is accepted and 1000.01 is not.

I also learned to check my diagrams against my code. When I compared my first flowchart with the program, two things did not match: the YES and NO outcomes of one decision were swapped, and the chart said "re-prompt" while the code sent the user back to the menu. I changed the code so it re-asks the field, redrew the flowcharts, and now the charts and the program agree.

9. Limitations of this phase

- Records live only in memory, so everything is lost when the program closes.
- The array holds at most 50 records.
- The rate is simply fat × 8.5. SNF is checked but does not change the price, and neither does the milk type.
- The auditor slip always prints "APPROVED FOR VAT STORAGE"; it does not judge quality yet.
- There is no update or delete, only Create and Read, which is what CA-1 asks for.

10. Use of AI Tools and References

I designed the MilkCollection class, the menu and the first version of the program myself. I used an AI assistant (Claude) for learning and clarification only: to review my first draft and point out problems (the program hanging on non-numeric input, validation that did not match my data dictionary, and the swapped flowchart labels), to suggest the input-checking helper functions, and to help organise and word this report. I compiled and tested the final program and I can explain every part of it.